GenAI- Clinical Intelligence

(Domain-Specialized AI Agents with Compliance Guardrails)

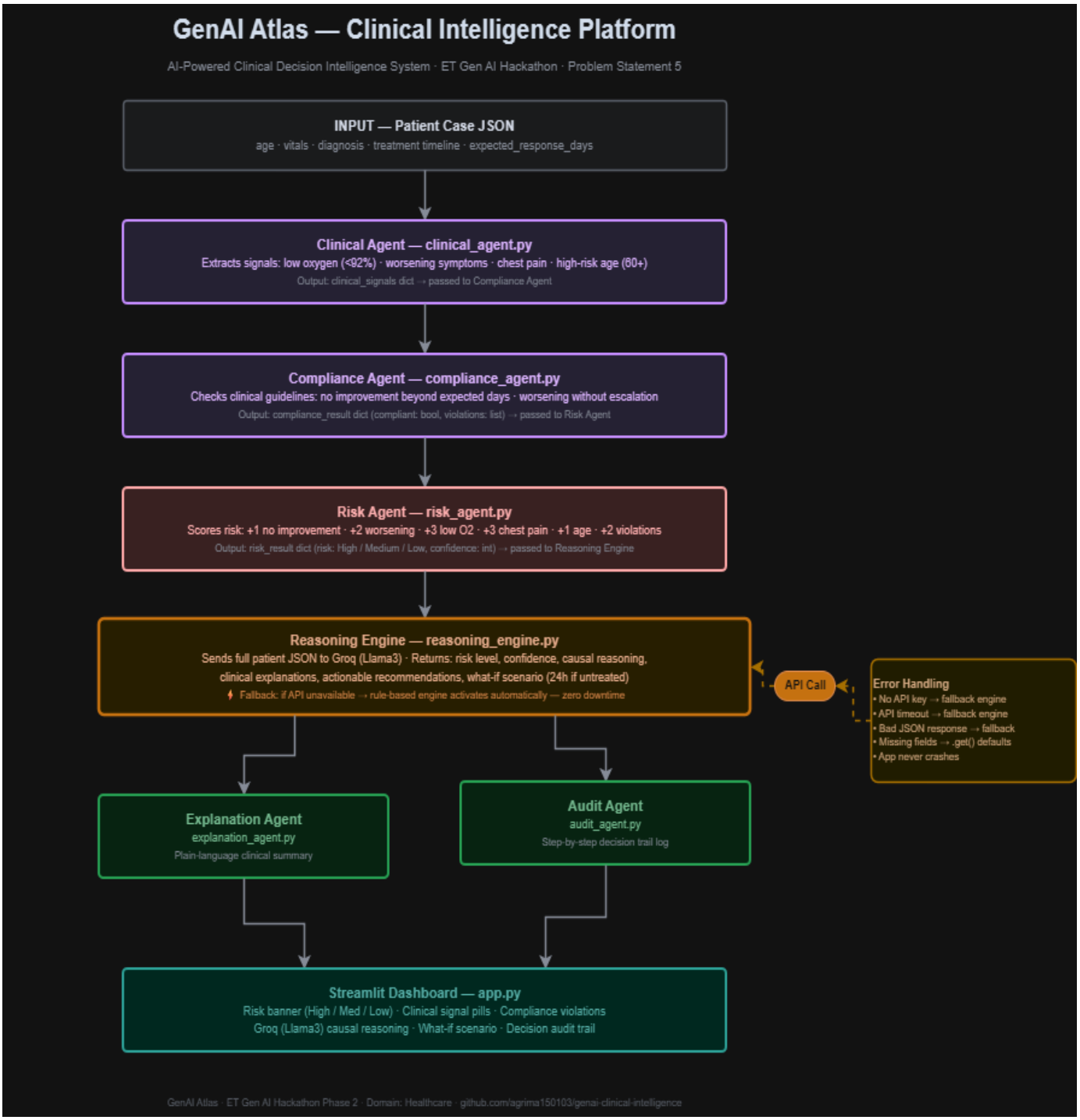

Diagram (draw.io): Healthcare operations

Architecture Description

1. Agent Roles

- **Clinical Agent (clinical_agent.py):**
  Parses the patient JSON and extracts clinically relevant signals: oxygen saturation below 92%, presence of worsening symptoms, chest pain and whether the patient is above age 60. It produces a structured signals dictionary used by all downstream agents.
- **Compliance Agent (compliance_agent.py):**
  Evaluates treatment against clinical guidelines using the signals from the Clinical Agent and checks whether the patient has failed to improve within the expected response window, whether symptoms are worsening without escalation and whether critical vitals were ignored. It produces a compliance result with a list of specific violations.
- **Risk Agent (risk_agent.py):**
  Calculates a numeric risk score by summing weighted signals: +1 for no improvement, +2 for worsening, +3 for low oxygen, +3 for chest pain, +1 for high-risk age, +2 for compliance violations. It classifies as High (score >= 6), Medium (>= 3), or Low, with a confidence percentage.
- **Reasoning Engine (reasoning_engine.py) — GenAI Layer:**
  The core GenAI component which sends the full patient case JSON to Groq (Llama3) via the Groq API with a structured prompt. Returns a JSON object containing: risk level, confidence score, list of clinical explanations, actionable recommendations, a causal reasoning paragraph and a what-if scenario describing consequences if untreated for 24 hours.
- **Explanation Agent (explanation_agent.py):**
  Translates structured agent outputs into plain language, clear, clinician friendly insights for intuitive decision making.
- **Audit Agent (audit_agent.py):**
  Logs every step of the decision pipeline, including detected signals, evaluated compliance rules, assigned risk levels and whether the LLM or fallback engine was used, creating a chronological audit trail.

2. How Agents Communicate

The agents operate as a sequential pipeline. Each agent receives two inputs: the original patient case JSON and the structured output of the previous agent. Data flows as Python dictionaries passed as function arguments, there is no message queue or broker.

The orchestration happens in app.py:

Patient JSON → clinical_signals → compliance_result → risk_result → LLM_result → explanation + audit → Dashboard

This design means each agent is independently testable and replaceable without affecting others.

3. Tool Integrations

Tool	Purpose	Integration point
Groq (Llama3)	LLM causal reasoning, what-if scenarios	reasoning_engine.py via groq Python library
Streamlit	Interactive web dashboard UI	app.py, all display logic
python-dotenv	Secure API key loading from .env file	reasoning_engine.py startup
JSON (stdlib)	Patient case data format	All agents read case_data dict

4. Error-Handling Logic

The system is designed to never crash, even in a production hospital environment:

- **Missing API key:** If GROQ_API_KEY is not set, the client is initialized as None. The analyze_case() function checks for this and immediately routes to the rule-based fallback engine.
- **API failure / timeout / rate limit:** The LLM call is wrapped in a try/except block. Any exception triggers the fallback engine and logs a warning, the app continues and always returns a result.
- **Malformed LLM response:** If Groq returns text that cannot be parsed as JSON, json.loads() fails and the fallback activates automatically. All required fields are defaulted via setdefault() before returning.
- **Missing patient fields:** All agents use .get() with safe defaults, no agent crashes on a missing key in the case JSON.